TECHNICAL SEMINAR REPORT & PRESENTATION GUIDE

SORA: Engineering an Autonomous On-Device AI Assistant

Architecture, Implementation, SLM Optimization & Full-Stack Deployment

Project Name:	Sora Personal AI Chatbot	**Target Platform:**	Native Windows 11 (64-bit)
Presenter / Author:	Sonu Chauhan	**Local Engine:**	Ollama v0.40.13 + llama.cpp
GitHub Repo:	codewithsonu19/Sora-personal-ai-chatbot	**Active Model:**	Qwen2.5:1.5B (Quantized Q4_K_M)

Seminar Abstract: Modern conversational AI assistants predominantly rely on cloud-hosted models (e.g. OpenAI GPT-4, Google Gemini, Anthropic Claude). While capable, cloud architectures entail recurring subscription fees, external latency dependencies, and severe privacy risks from transmitting sensitive personal data off-premise. This seminar presents **Sora**, a fully autonomous personal AI assistant engineered to run 100% locally on commodity PC hardware. This report explores the end-to-end technical implementation: low-level package orchestration, hardware topology discovery, local model calibration, FastAPI asynchronous server architecture, Server-Sent Events (SSE) token streaming, and an empirical performance investigation that reduced query latency from 6 minutes to 0.66 seconds (a 550x speedup).

1. Core Problem Statement & Seminar Objectives

- **The Data Privacy Vacuum:** Cloud chatbots store and analyze user transcripts. Sora guarantees absolute zero data exfiltration by terminating all token generation inside local RAM.
- **Democratizing Edge AI:** Demonstrating that lightweight Small Language Models (SLMs) can deliver conversational fluency, multi-tool calculation, and reasoning without expensive discrete GPUs.
- **Full-Stack Decoupled Design:** Bridging native OS primitives with web standards via Python FastAPI and React 19.
- **Empirical Performance Engineering:** Demonstrating the exact techniques used to profile, debug, and resolve memory bus and prompt evaluation bottlenecks.

2. System Architecture & High-Level Design

Sora is designed with a decoupled, four-tier reactive architecture. Decoupling ensures that client rendering, REST API routing, agent orchestration, and tensor inference run in dedicated process boundaries without cascading failures.

System Tier	Components & Implementation	Core Responsibilities & Capabilities
Tier 1: Client UI	React 19, TypeScript, Vite 8, Tailwind CSS, Lucide Icons	Browser chat dashboard at localhost:5173, real-time token rendering, audio read-aloud playback, model switcher, dark/light theme engine.
Tier 2: API Gateway	FastAPI ASGI, Uvicorn, Python 3.12, Pydantic v2	OpenAI-compatible endpoints (/v1/chat/completions, /health), Server-Sent Events (SSE) streaming generator, request middleware.
Tier 3: Agent & Reasoning	OpenJarvis Orchestrator, Multi-agent Router, Tool Executor	Autonomous tool invocation (calculator, think), loop-guard safety checks, persona grounding via SOUL.md, contextual history tracking.
Tier 4: Local Inference Engine	Ollama Daemon (v0.40.13), llama.cpp backend, Qwen SLMs	Headless LLM daemon on port 11434, GGUF tensor dequantization, dynamic KV cache allocation, multi-thread CPU execution.
Tier 5: Speech & Native Stubs	CTranslate2, Faster-Whisper, PyO3/Rust Stubs	On-device Speech-to-Text (STT) voice transcription using INT8 quantized Whisper-base; low-overhead native memory bindings.

Component Interaction Workflow

1. **User Input:** User types a query into the React web UI or speaks via the browser microphone.
2. **Event Dispatch:** The browser opens an SSE POST stream to http://127.0.0.1:8000/v1/chat/completions.
3. **Context & Persona Augmentation:** FastAPI injects the local persona prompt from ~/.openjarvis/SOUL.md.
4. **Inference Evaluation:** The Ollama engine evaluates the prompt via llama.cpp across multi-core CPU threads.
5. **Streaming Tokens:** Tokens stream back word-by-word through SSE deltas and render instantaneously on screen.

3. Detailed Technology Stack & Technical Specifications

The following matrix summarizes the exact technologies, version specifications, and architectural roles utilized in the Sora personal chatbot deployment:

Category	Technology / Tool	Version	Technical Justification & Role
Runtime	Python	3.12.8	Core runtime. Avoided Python 3.14 due to missing Windows wheels for numpy/torch.
Package Manager	uv (Astral)	0.13.6	High-speed Rust-based package manager. Eliminates dependency resolution overhead.
LLM Engine	Ollama Runtime	0.40.13	Headless C++ daemon managing weights, GGUF loading, and tensor operations.
Primary Model	Qwen2.5:1.5B	1.54B (Q4_K_M)	Selected for ultra-low latency, CPU efficiency, and absence of hidden thinking loops.
Reasoning Model	Qwen3.5:2B	2.0B (Q8_0)	Evaluated for deep multi-turn chain-of-thought analysis and tool-orchestration tests.
API Server	FastAPI	0.136.3	Asynchronous web framework delivering high-throughput OpenAI-compatible endpoints.
ASGI Server	Uvicorn	0.54.0	Production ASGI server running the async HTTP / EventSource loop.
Frontend Tooling	Vite	8.3.4	Sub-second Hot Module Replacement (HMR) and optimized client production bundling.
UI Framework	React	19.0.0	Declarative component architecture for streaming chat bubbles and telemetry widgets.
Language	TypeScript	5.7	Full type safety across frontend stores, SSE event shapes, and REST API models.
Speech Backend	Faster-Whisper	1.2.1	CTranslate2-powered transcription model running locally in INT8 compute mode.
Version Control	Git + GitHub	2.53.0	Automated repository syncing, version tracking, and remote origin deployment.

Technical Note on Packaging: The project utilizes Astral's uv package manager rather than legacy pip. uv resolved 454 transitive dependencies and built the entire desktop environment in 61 seconds. It enforces deterministic lockfiles via uv.lock, guaranteeing bit-for-bit reproducibility across machines.

4. Step-by-Step Implementation & Deployment Blueprint

This section serves as a reproducible implementation guide. Follow these phases to build, configure, and launch the assistant:

Phase 1: Environment & Prerequisite Audit

1. Detect OS and active shell: Windows 11 Home (Build 10.0.26300, 64-bit), Native PowerShell.
2. Verify Python version: python --version (Must be ≥ 3.10 and ≤ 3.13; strictly avoiding 3.14).
3. Verify Node.js runtime: node --version (Must be ≥ v22.0.0; detected v24.21.0).
4. Disk Space Check: Verified 30.96 GB free on C: and 294 GB on D: (exceeds the 5 GB minimum requirement).

Phase 2: Package Orchestration & Engine Bootstrapping

1. Install uv: irm https://astral.sh/uv/install.ps1 | iex
2. Set ExecutionPolicy: Set-ExecutionPolicy RemoteSigned -Scope CurrentUser -Force
3. Sync environment: uv sync --extra desktop (Builds 81 packages including FastAPI, Whisper, Uvicorn).
4. Install Ollama: Download OllamaSetup.exe and execute silently: Start-Process OllamaSetup.exe -ArgumentList '/VERYSILENT /NORESTART /SP-' -Wait
5. Start engine daemon: Start-Process ollama.exe -ArgumentList 'serve' -WindowStyle Hidden

Phase 3: Identity Architecture & Persona Injection ('Sora')

1. Write Agent Persona: Update ~/.openjarvis/SOUL.md with: You are Sora, a helpful personal AI assistant.
2. Configure Engine & Model: In ~/.openjarvis/config.toml, bind [engine] default = 'ollama' and default_model = 'qwen2.5:1.5b'.
3. CLI Wordmark: Update src/openjarvis/cli/_banner.py to render the ASCII wordmark for **Sora**.
4. Web UI Branding: Update frontend/index.html (<title>Sora</title>) and InputArea.tsx (Message Sora...).

Phase 4: Diagnostics, Multi-Agent Tool Checks & Launch

1. Run jarvis doctor: Confirmed 10 passed checks (Python 3.12, Ollama Reachable, Whisper Ready, API Installed).
2. Tool test: jarvis ask --agent orchestrator --tools calculator,think "What is 137 * 137 * 42?" → Verified 788,298.
3. Start API Server: uv run jarvis serve --port 8000 → Health probe returns {'status': 'ok'}.
4. Start Web UI: cd frontend && npm run dev → Live at http://localhost:5173.
5. Remote Sync: Synchronized git database (0 missing objects) and pushed to github.com/codewithsonu19/Sora-personal-ai-chatbot.

5. Performance Engineering Case Study: Resolving the 6-Minute Delay

A critical engineering obstacle encountered during deployment was an unacceptable query latency of **over 6 minutes (369.8 seconds)** for simple questions. This case study details the empirical investigation, root-cause diagnosis, and technical solution.

Identified Bottleneck	Underlying Technical Mechanism	Applied Engineering Resolution
1. Hidden Thinking Loop	qwen3.5:2b had thinking: default true with uncompressed Q8_0 weights, emitting **4,076 hidden <think> tokens** before generating visible text.	Migrated to **Qwen2.5:1.5B** (986 MB, Q4_K_M). Designed for immediate conversational responses without hidden thinking loops.
2. Excessive Context Window	Context window defaulted to **16,384 tokens**. On CPU without high-bandwidth VRAM, calculating the attention matrix for 16k tokens created extreme RAM bus contention.	Configured JARVIS_NUM_CTX = 2048 in ollama.py, shrinking the KV-cache footprint by ~80% and slashing prompt eval time.
3. Blocking Server Buffering	In routes.py, requests with tools were routed through a synchronous worker thread that buffered the entire answer before sending it.	Enabled **direct SSE token-by-token streaming** in routes.py, streaming tokens to the browser immediately as they are generated.

Empirical Benchmark Results

Quantitative measurements confirmed a **557x reduction in total query turnaround time** on the same host CPU:

Benchmark Metric	Initial State (Unoptimized)	Final State (Optimized Sora)	Net Performance Gain
Model & Quantization	qwen3.5:2b (2.7 GB, Q8_0)	qwen2.5:1.5b (986 MB, Q4_K_M)	63% smaller footprint
Context Window (KV Cache)	16,384 tokens	2,048 tokens	8x faster KV allocation
Hidden Reasoning Overhead	4,076 <think> tokens	0 tokens (immediate answer)	Zero wasted compute
Total Response Latency	369,854 ms (6.1 minutes)	663 ms (0.66 seconds)	**557x Faster Turnaround**
Time to First Token (TTFT)	> 30,000 ms	< 800 ms	Instant conversational response
Token Generation Throughput	11.3 tokens/sec	22.4 tokens/sec	2x higher generation speed

Seminar Takeaway: *'Running AI locally on standard PCs is not restricted by raw compute alone, but by memory bus bandwidth. Allocating oversized context windows and triggering unconstrained reasoning loops can silently exhaust hardware capacity. Co-designing context limits and using quantized small language models unlocks real-time interactivity on everyday laptops.'*

6. Seminar Presentation Talking Points & Structure

When presenting this project in the seminar hall, organize your presentation slide deck and delivery around these four thematic pillars:

1. **Slide Topic — The Rise of On-Device AI:** Open by highlighting that cloud AI models cost organizations thousands of dollars monthly while leaking private context. Contrast this with Sora's $0 operating cost and 100% offline security.
2. **Slide Topic — Software Architecture Breakdown:** Walk the audience through the multi-tier design: how Vite 8 + React 19 handle UI state, how FastAPI coordinates endpoints, and how Ollama drives the llama.cpp engine.
3. **Slide Topic — The Latency Breakthrough:** Present the 370s → 0.66s latency curve. Explain how profiling memory bus bandwidth and disabling hidden reasoning loops transformed the assistant from unusable to real-time.
4. **Slide Topic — Live Demonstration:** Showcase the live browser UI at http://localhost:5173 and the terminal CLI asking complex multi-tool queries.

7. Future Roadmap & Architectural Extensions

Feature Area	Proposed Technical Extension	Anticipated Capability & Impact
Local RAG	Integrate ChromaDB / SQLite vector embeddings for local document retrieval (PDF, Word, Code).	Allows Sora to synthesize personal knowledge bases without cloud embedding APIs.
Vision Multimodality	Deploy quantized vision projectors (e.g. Moondream2 or SmolVLM) via GGUF.	Enables local OCR, visual diagram analysis, and image question-answering.
Hardware Acceleration	Integrate Intel OpenVINO / DirectML drivers for Intel UHD / Iris Xe integrated graphics.	Offloads matrix multiplications from CPU to GPU Execution Units (EUs) for an additional 2x boost.

Conclusion & Key Takeaway: The Sora personal AI chatbot demonstrates that enterprise-grade AI assistants no longer require massive cloud server farms. Through intelligent model selection, rigorous context tuning, and modern asynchronous web protocols, high-speed, secure, and autonomous AI can be deployed on standard personal computing hardware.